Boudrouss Réda | CV

Software Engineer – OCaml, WebAssembly & Static Analysis – Paris

github.com/rboudrouss
linkedin.com/in/rboudrouss
contact@rboud.com
rboud.com
+33 (0)7 55 78 85 16

Languages: French (Native), English (Professional), Arabic (Conversational)

Profile

Software engineer completing an MSc in Computer Science (Software Science & Technology) at Sorbonne University, specialized in **static analysis**, **abstract interpretation** and the implementation of **programming languages** and their runtimes. I ported **MOPSA**, an OCaml/C/C++ abstract-interpretation analyzer, to **WebAssembly**, which took me deep into the OCaml **runtime**, its C FFI and float semantics on Wasm. The result is a fully client-side browser playground, documented in a public write-up.

Projects

Mopsa-wasm - github.com/rboudrouss/mopsa-emcc 2026

· Compiled the **OCaml runtime** and the full native stack of **MOPSA** (Clang frontend, Apron, GMP, MPFR) into a single static **WebAssembly** module with Emscripten, an approach chosen over `js_of_ocaml` and `wasm_of_ocaml` because it brings the C/C++ half of the codebase along.

· Replaced the **runtime**'s dynamic FFI loading with a generated static table of ∼**1,400 C primitives**, and root-caused an unsigned-wraparound bug in the `Tag_val` macro, a native-vs-Wasm semantics mismatch that traps under Wasm's bounds-checked addressing.

· Addressed the **soundness** hazard of Wasm's fixed rounding mode (no `fesetround`) by recompiling Apron's numerical domains to use exact GMP rationals.

· Shipped a fully client-side playground (Web Workers around a non-reentrant runtime, blocking stdin over SharedArrayBuffer + Atomics for the interactive REPL and DAP modes) and a technical write-up.

Reactant analyzer - github.com/rboudrouss/reactant-analyzer 2026

· Designed and implemented an **abstract-interpretation**-based static analyzer in **Rust** for React web applications.

Education

MSc in Computer Science (Software Science & Technology) - Sorbonne University 2024 – 2026

· Coursework: compiler construction, design & implementation of **programming languages** and their runtimes (interpreters vs. compilers), formal semantics, **type systems** and **static analysis by abstract interpretation** (course led by Antoine Miné), concurrent & distributed programming.

BSc in Computer Science - Sorbonne University 2020 – 2024

· Algorithms, computer architecture, operating systems, networks, databases, functional and object-oriented programming, with team projects in C, Java, OCaml, Python and TypeScript.

Experience

Quality Analyst Engineer (apprenticeship) - Hexaglobe (video streaming) 2025 – 2026

· Designed complete **test strategies** (unit, integration, E2E) for legacy products with no prior coverage, reaching ∼**3,000 test cases** across 4 stacks (TypeScript, Python, PHP, JS), partly through AI-agent test production under versioned guardrails and systematic human review.

· Built objective **quality oracles** (VMAF, black-frame/freeze detection, audio analysis) with ffmpeg to validate a live video encoder.

· Built complete **GitLab CI/CD** pipelines and shared CI components (npm publishing, centralized Allure reporting) consumed by 4+ repositories.

Technical Lead, then President (2024–25) - Association Play Sorbonne Université 2023 – 2026

· Designed and operated a virtualized **infrastructure** (Docker servers, CI/CD, networking) and built a **local-first** mobile web app. As president, led **100+ volunteers** for a cultural event welcoming 10,000+ visitors.

Skills

Languages: OCaml, Rust, C, TypeScript, Python, Java, PHP, SQL
Topics: WebAssembly & Emscripten, static analysis & abstract interpretation, compilers & runtimes, GitLab CI/CD, Docker